

CONTEXT-AWARE CODING AGENT

12-Hour Build & Submission Playbook

Beginner-friendly, step-by-step execution guide

THE ONE RULE

Do not move to the next phase just because the code exists. Move only when the checkpoint works. The blueprint explicitly recommends completing each demoable phase before starting the next.

What you need to finish	Priority	Target
Phases 1-6: working MVP	NON-NEGOTIABLE	First priority
Phases 7-8: related context + structured agent communication	HIGH	Do after MVP
Phase 9: polished sidebar/webview UI	MEDIUM	Only after core flow works
Phase 10: packaging, settings, hardening	FINAL	Before submission

Use this PDF like a lab manual: read one step, do it on the computer, check the expected result, then continue.

How to use this guide

- [] Keep this PDF open beside VS Code while building.
- [] Never paste a later-phase feature into the project just because it looks impressive. The MVP is the safe file-editing loop, not the UI.
- [] When a step says "Expected result", stop and verify it before moving on.
- [] When a command fails, use the troubleshooting section instead of randomly changing packages or architecture.
- [] Keep the original Project Blueprint nearby. This guide expands its execution steps; it does not replace the blueprint.

Contents

1. 12-hour survival schedule
2. What the final project must do
3. Step 0 - Computer and software setup
4. Step 1 - Generate the VS Code extension
5. Step 2 - Install dependencies
6. Step 3 - Create the project folder structure
7. Phase 1 - Extension skeleton and active-file reader
8. Phase 2 - File-specific memory
9. Phase 3 - Local Qwen via Ollama
10. Phase 4 - Gemini provider
11. Phase 5 - Safe code edits and diff approval
12. Phase 6 - Validation and correction loop
13. Phase 7 - Related-file context
14. Phase 8 - Context packets and agent communication
15. Phase 9 - Sidebar/webview UI
16. Phase 10 - Packaging, hardening and submission
17. Full end-to-end test script
18. Troubleshooting checklist
19. Final submission checklist

TEAM EDITION - 4 PEOPLE + IBM BOB

WORKFLOW

This version assumes all four teammates are beginners and IBM Bob is doing the actual coding. Your job is to operate the terminal, give Bob clear implementation tasks, run tests, review what Bob produced, and collect real evidence of completed work.

A. Golden Rule: Nobody Needs To Manually Write The Extension Code

- 1. Use IBM Bob to create or modify the TypeScript files described in the blueprint.
- 2. You and the team run only the required terminal commands shown in this guide.
- 3. After Bob changes something, run the stated verification command before moving on.
- 4. Save a screenshot or terminal result when a task is completed. This is genuine work evidence, not a fake activity log.
- 5. If Bob proposes a change that conflicts with the blueprint safety rule, stop and ask Bob to correct it before continuing.

B. 4-Person Work Split

Person	Main ownership	What they actually do	Evidence to save
YOU (Lead, full-time)	Integration + Bob driver	Own the main task list; give Bob the architecture/phase task; run setup, dependencies, provider integration, final integration and packaging; decide what moves to next phase.	Terminal screenshots, Bob task prompts, successful tests, final .vsix build, final demo
SHUBHAM (Full-time)	Memory + testing partner	Work alongside you on Phase 2 memory, Qwen setup, test cases, debugging with Bob, and checking that features work after restart.	Memory JSON screenshots, test outputs, bug/fix screenshots, restart verification
SACHIN (When available)	QA + UI/support	When online, run one focused task at a time: prerequisite checks, Qwen/Gemini connectivity checks, manual test cases, diff approval test, sidebar smoke test.	Terminal result + screenshot for each completed test; short issue/fix note
RAM (light but real workload)	Documentation + evidence + release checks	Maintain README, take architecture/demo screenshots, run the final checklist, verify VSIX installation, record known limitations and submission files.	README commit/file, screenshot set, VSIX install proof, final checklist, demo notes

C. 12-Hour Team Schedule

Time	YOU	SHUBHAM	SACHIN	RAM
0-1 h	Install/check tools; create repo	Install/check tools; VS Code ready	When available: verify Node/Git/VS Code	Create evidence folders + README skeleton
1-3 h	Scaffold extension; drive Bob through Phase 1	Drive Bob through memory + hashing; test file memory	Run Phase 1 smoke test when available	Capture screenshots + update README setup
3-5 h	Qwen + Gemini provider setup with Bob	Qwen memory compression + provider checks	Provider connectivity smoke tests	Document provider setup and limitations
5-7 h	Diff/approval integration + orchestrator	Validation/test loop + bug reproduction	Run approve/reject + failed-validation tests	Security checklist + evidence organization
7-9 h	Related-file context + integration	Context packet / token-filter checks	Sidebar/UI smoke test when available	Demo script + screenshots

9-10.5 h	Full integration and bug triage	Full end-to-end tests	Retest any fixed issues	README + known limitations
10.5-12 h	Package + final demo	Clean install + final test	Final smoke test	VSIX install proof + final submission folder

D. Evidence Folder - Create This At The Start

Use this only for genuine completed work. Each person should save screenshots/results from the tasks they actually performed.

Mac Terminal: `mkdir -p evidence/you evidence/shubham evidence/sachin evidence/ram`

Windows PowerShell: `New-Item -ItemType Directory -Force -Path evidence\you,evidence\shubham,evidence\sachin,evidence\ram`

Suggested evidence names: 01-setup.png, 02-phase1.png, 03-memory-test.png, 04-provider-test.png, 05-diff-approval.png, 06-validation.png, 07-vsix-install.png, 08-final-demo.png

E. Terminal-Only Command Reference

The commands below are the operations YOU run. They work the same on macOS Terminal and Windows PowerShell unless a platform label is shown.

Purpose	macOS Terminal	Windows PowerShell
Check Node + npm	<code>node --version</code> <code>npm --version</code>	<code>node --version</code> <code>npm --version</code>
Check Git	<code>git --version</code>	<code>git --version</code>
Check VS Code CLI	<code>code --version</code>	<code>code --version</code>
Check Ollama	<code>ollama --version</code>	<code>ollama --version</code>
Install generator	<code>npm install --global yo generator-code</code>	<code>npm install --global yo generator-code</code>
Create project	<code>mkdir -p context-aware-extension</code> <code>cd context-aware-extension</code>	<code>mkdir context-aware-extension</code> <code>cd context-aware-extension</code>
Run scaffold	<code>yo code</code>	<code>yo code</code>
Open in VS Code	<code>code .</code>	<code>code .</code>
Install core packages	<code>npm install zod fast-glob ignore diff</code> <code>tiktoken</code>	<code>npm install zod fast-glob ignore diff</code> <code>tiktoken</code>
Install Git helper (optional)	<code>npm install simple-git</code>	<code>npm install simple-git</code>
Install parser packages (later)	<code>npm install tree-sitter tree-sitter-typescript</code> <code>tree-sitter-javascript</code>	<code>npm install tree-sitter tree-sitter-typescript</code> <code>tree-sitter-javascript</code>
Install dev/test packages	<code>npm install --save-dev typescript esbuild</code> <code>@types/node @types/vscode</code> <code>npm install --save-dev eslint mocha</code> <code>@vscode/test-cli @vscode/vsce</code>	<code>npm install --save-dev typescript esbuild</code> <code>@types/node @types/vscode</code> <code>npm install --save-dev eslint mocha</code> <code>@vscode/test-cli @vscode/vsce</code>
Type-check	<code>npm run check-types</code>	<code>npm run check-types</code>
Lint	<code>npm run lint</code>	<code>npm run lint</code>
Tests	<code>npm test</code>	<code>npm test</code>
Package VSIX	<code>npm run package</code>	<code>npm run package</code>

Important: Do not paste TypeScript code from the guide into files. When a phase says "create/update this file", give the phase instructions to IBM Bob and let Bob implement it. Your terminal job is to install, run, verify, and package.

F. How To Give IBM Bob A Task

For every phase, send Bob three things: (1) the phase goal, (2) the exact files from the blueprint that should be created/updated, and (3) the acceptance test from this guide. Ask Bob to make the change, then immediately run the terminal verification command listed here.

Example workflow: Bob implements Phase 2 memory -> you run `npm run check-types` -> Shubham opens two files and tests independent memory -> Ram records the result -> only then move to Phase 3.

G. Beginner Rule For The Four Of You

Never debug by changing five things at once. Change one thing, run one verification command, record the result, and move on. If Bob gets stuck, copy the terminal error into Bob and ask for a targeted fix.

1. The 12-Hour Survival Schedule

IF YOU ARE RUNNING OUT OF TIME
Stop at Phase 6, package the MVP, test it, and submit. The supplied blueprint defines Phases 1-6 as the MVP. UI polish is not more important than a working diff -> approval -> write -> validation loop.

Time from start	Do this	Do not do yet
0:00-0:30	Node/npm/Git/VS Code/Ollama setup; generate extension	Do not start UI
0:30-1:30	Phase 1: active file reading + one command	No Gemini yet
1:30-2:45	Phase 2: memory file, file IDs, hashes, events	No fancy memory compression
2:45-4:00	Phase 3: Qwen provider + summaries	No repo-wide scanning
4:00-5:30	Phase 4: Gemini provider + structured JSON	No multi-provider architecture
5:30-7:15	Phase 5: diff, scope validation, WorkspaceEdit approval	No autonomous writes
7:15-8:30	Phase 6: type-check/lint/tests + max 2-3 correction attempts	No infinite loops
8:30-9:15	Security hardening + secret filtering	No public publishing
9:15-10:15	Phase 7-8 if MVP is stable	Do not break MVP
10:15-11:00	Phase 9 UI only if time remains	Do not redesign architecture
11:00-11:30	Phase 10 package as VSIX	No risky refactors
11:30-12:00	Full demo test, README, screenshots/video, final folder check	No new features

Blueprint basis: Section 10 "Build Roadmap" and the Definition of Done for the MVP (Phases 1-6).

2. What You Are Actually Building

The finished extension is a VS Code coding assistant that works on one active file at a time, keeps a separate memory record for that file, uses a local Qwen model for cheap context work, uses Gemini for harder reasoning/code generation, and never writes an AI change directly to disk without showing a diff and getting approval.



```

      v
Type-check / lint / tests
      |
      v
Memory update + Qwen compression

```

DO NOT BREAK THIS RULE

The AI proposes JSON. The extension validates it. VS Code shows a diff. Only after explicit approval does WorkspaceEdit apply the change.

3. Step 0 - Prepare the Computer

1. Open a terminal

On Windows, open PowerShell or VS Code -> Terminal -> New Terminal.

2. Check Node.js and npm

Run these commands. If each returns a version number, continue.

Type / create exactly this:

```
node --version
npm --version
```

Expected result: You see a Node.js version and an npm version.

3. Check Git

Git is the project history/rollback tool in the blueprint.

Type / create exactly this:

```
git --version
```

Expected result: You see a Git version.

4. Check VS Code

Open VS Code normally. You will later press F5 to launch the Extension Development Host.

5. Install/prepare Ollama

Install Ollama on the machine before Phase 3. The blueprint uses Qwen through Ollama as the local model provider.

DO NOT HIDE KEYS IN CODE

Gemini credentials belong in VS Code SecretStorage (context.secrets), not source files or settings.json. This is a submission requirement, not a later optimization.

4. Step 1 - Generate the VS Code Extension

1. Install the project generator

Run the exact scaffold command from the blueprint.

Type / create exactly this:

```
npm install --global yo generator-code
```

2. Create the project folder

Use a short path so Windows path issues are less likely.

Type / create exactly this:

```
mkdir context-aware-extension
cd context-aware-extension
```

3. Run the generator

The generator asks questions. Choose a TypeScript extension with esbuild and npm.

Type / create exactly this:

```
yo code
```

4. Open the project in VS Code

From the project folder:

Type / create exactly this:

```
code .
```

Expected result: The scaffold opens in VS Code.

5. Run the empty extension

Press F5. A new "Extension Development Host" VS Code window should appear.

Expected result: The development host opens without a compilation error.

Blueprint basis: Section 13 Appendix, scaffold commands. The blueprint specifies TypeScript + esbuild + npm.

5. Step 2 - Install Dependencies

1. Install core runtime dependencies

These support validation, file discovery, ignore rules, diffs, and token counting.

Type / create exactly this:

```
npm install zod fast-glob ignore diff tiktoken
```

2. Install optional Git awareness

The blueprint marks simple-git optional, so skip it if time is tight.

Type / create exactly this:

```
npm install simple-git
```

3. Install later structural parsing dependencies

Tree-sitter is optional and later. Do not let it block the MVP.

Type / create exactly this:

```
npm install tree-sitter tree-sitter-typescript tree-sitter-javascript
```

4. Install development/testing/publishing dependencies

These are for TypeScript checking, bundling, linting, VS Code tests, and VSIX packaging.

Type / create exactly this:

```
npm install --save-dev typescript esbuild @types/node @types/vscode
npm install --save-dev eslint mocha @vscode/test-cli @vscode/vsce
```

Blueprint basis: Section 3 and Appendix dependencies.

6. Step 3 - Create the Project Structure

Create the folders and files below. Empty files are fine at first. Their responsibilities are defined in the next phases.

```
src/
  extension.ts
  commands/
    askForChange.ts
    explainFile.ts
    showMemory.ts
    clearFileMemory.ts
    reviewChange.ts
  orchestrator/
    AgentOrchestrator.ts
    TaskClassifier.ts
    WorkflowState.ts
    AgentMessageBus.ts
  ai/
    AiProvider.ts
    GeminiProvider.ts
    QwenProvider.ts
    ProviderRouter.ts
    AiSchemas.ts
  memory/
    MemoryStore.ts
    MemorySchema.ts
```

```

FileIdentity.ts
MemoryUpdater.ts
MemoryCompressor.ts
context/
  ActiveFileReader.ts
  RelatedFileResolver.ts
  ContextBuilder.ts
  ContextFilter.ts
  TokenBudget.ts
  SecretFilter.ts
artifacts/
  ArtifactStore.ts
  ContextPacket.ts
  ArtifactSchema.ts
changes/
  ChangeParser.ts
  DiffPreview.ts
  ChangeValidator.ts
  WorkspaceEditor.ts
validation/
  TestRunner.ts
  LintRunner.ts
  ValidationResult.ts
utils/
  hashing.ts
  logging.ts
  paths.ts

test/
.vscode/
package.json
tsconfig.json
esbuild.js
README.md
.gitignore

```

KEEP MVP SMALL

You do not need to fully implement every file above before you can demonstrate the project. Build the Phase 1-6 path first. The later files can stay minimal until their phase.

7. Phase 1 - Extension Skeleton + Active File Reader

Goal: open a file -> run a command -> read the live editor content -> show a simple summary. This is the first demoable deliverable.

1. Create `ActiveFileReader.ts`

Read the current text editor, URI, relative path, language ID, and live text. Use the editor buffer, not only saved disk content.

2. Create `explainFile.ts`

Add a command that calls `ActiveFileReader` and reports a small summary in an output panel or VS Code information message.

3. Register the command in `extension.ts`

The extension activation function should register the command and call `explainFile` when the user triggers it.

4. Add the command to `package.json`

The command ID must be declared so it appears in the Command Palette. Use a clear title such as "Context Agent: Explain Current File".

5. Test it

Open a TypeScript file with unsaved changes. Run the command. Confirm the output reflects the current editor text.

PHASE 1 CHECKPOINT

STOP. If you cannot open a file and see its current path/language/content summary, do not start memory or AI work yet.

8. Phase 2 - File-Specific Memory

Goal: every source file gets its own persistent memory record. The memory survives VS Code restarts and is updated instead of replaced.

1. Create .agent_memory.json

This is the persistent workspace-level memory file. The blueprint stores separate records inside it, keyed by a stable file ID.

```
.agent_memory.json
{
  "version": 1,
  "workspaceId": "workspace-abc123",
  "files": {}
}
```

2. Create MemorySchema.ts

Define the exact shape for workspace memory, file records, memory events, constraints, decisions, and known flaws.

```
FileMemory = {
  fileId,
  currentHash,
  language,
  summary,
  constraints: [],
  decisions: [],
  knownFlaws: [],
  events: []
}
```

3. Create FileIdentity.ts

Generate a stable fileId and keep both path and identity. Do not rely only on path because files can be renamed.

4. Create hashing.ts

Hash the current file content. Same hash = no content change; changed hash = update the existing memory record and append an event.

5. Create MemoryStore.ts

Implement get(fileId), update(fileId, event), remove(fileId), and save(). Read/write .agent_memory.json through the VS Code file system API.

6. Save atomically

Write to .agent_memory.json.tmp first, then rename it over the real memory file. This prevents a partial write from corrupting the main file.

7. Create showMemory.ts

Add a command that shows the active file memory so you can prove the feature works in a demo.

8. Create clearFileMemory.ts

Add a safe command to deliberately delete/reset the active file memory when testing.

9. Test persistence

Open file A, create memory, switch to file B, create different memory, restart VS Code, and confirm both records survive.

PHASE 2 CHECKPOINT

You should be able to prove that file A and file B have different memory, and that memory remains after restarting VS Code.

Blueprint basis: Section 5 File-Specific Memory System and Section 11 team plan for MemoryStore.

9. Phase 3 - Local Qwen via Ollama

Goal: Qwen handles cheap, frequent context work locally: classification, summarization, compression, filtering, and extraction of imports/symbols.

1. Choose a model size appropriate to your hardware

The blueprint suggests 0.6B-4B for lower-spec hardware, 7B-8B for mid-range hardware, and 14B+ for strong GPUs. Do not choose a model that is too slow to be useful.

2. Verify Ollama is running

Use the Ollama application/service and verify a local model can answer a simple prompt before connecting it to the extension.

3. Create AiProvider.ts

Define a provider interface so the orchestrator can call Qwen and Gemini through a common shape.

4. Create QwenProvider.ts

Implement the local HTTP call to Ollama. Start with one method: summarize the active file.

5. Test from the extension

Run Explain Current File and use Qwen to produce a short summary.

6. Add memory compression

Pass the active file summary plus relevant recent events to Qwen and ask it for a compact summary, current decisions, and known flaws.

7. Store summaries, not transcripts

Do not save long raw conversations in memory. Save intent, decisions, changed symbols, bugs, corrections, and validation results.

PHASE 3 CHECKPOINT

A file can be summarized locally and its memory can be compressed locally. The extension should work even when the cloud model is not being called.

Blueprint basis: Section 6 AI Provider Strategy and Phase 3 of the build roadmap.

10. Phase 4 - Gemini Provider

Goal: Gemini receives a compact context packet and returns a structured proposed change. It should not receive the whole repository.

1. Get a Gemini API credential for development

The blueprint warns not to assume a consumer Google subscription automatically provides Gemini API quota. Set up API credentials separately for development.

2. Store the credential in SecretStorage

Use VS Code context.secrets. Never put the key in source, README, settings.json, or logs.

3. Create GeminiProvider.ts

Implement the API call and structured-response handling. Keep the provider behind AiProvider.ts.

4. Create AiSchemas.ts

Use Zod to validate the response before any file is changed.

```
Expected response shape
{
  updatedFile: string,
  summary: string,
  decisions: string[],
  warnings: string[],
  testsToRun: string[],
  confidence: number
}
```

5. Make Gemini generate a proposed change

Based on the supplied Project Blueprint | MVP = Phases 1-6 | Full build = Phases 1-10

Start with a simple request such as: "Add input validation to this function." The output must be structured JSON containing the full proposed file or the supported patch representation.

6. Reject invalid responses

If JSON is malformed, required fields are missing, or the target file is outside the allowed scope, stop before writing anything.

PHASE 4 CHECKPOINT

Gemini can receive a controlled input and return a validated structured response representing a proposed change. No disk write happens yet.

Blueprint basis: Section 3 AI layer, Section 6 provider strategy, and Phase 4 deliverable.

11. Phase 5 - Safe Code Edits and Diff Approval

Goal: the extension makes it technically impossible for an AI response to silently overwrite a file.

1. Create ChangeParser.ts

Turn the validated AI response into an internal change object. Keep the target file ID/path attached to the change.

2. Create ChangeValidator.ts

Check that the response touches only the file(s) explicitly in scope, is reasonable in size, and passes Zod validation.

3. Create DiffPreview.ts

Compare original content with proposed content using the diff dependency and display the result using the VS Code diff editor or an equivalent review UI.

4. Add Approve and Reject actions

Reject = discard proposal. Approve = continue to WorkspaceEditor. There must be no hidden auto-approval path.

5. Create WorkspaceEditor.ts

Use VS Code WorkspaceEdit to apply the approved edit as an all-or-nothing editor operation, rather than directly overwriting the file with a raw write.

6. Test rejection

Request a real change, review the diff, click Reject, and verify the file remains unchanged.

7. Test approval

Repeat the same test, click Approve, and verify the file changes exactly as shown in the diff.

PHASE 5 CHECKPOINT

You can demonstrate: AI proposal -> diff -> explicit user approval -> file update. Before approval, the file must remain untouched.

Blueprint basis: Section 2 design rule, Section 8 steps 8-10, and Phase 5 deliverable.

12. Phase 6 - Validation and Correction Loop

Goal: every approved change is validated after it is applied. Failures can be sent back to Gemini for a limited correction pass.

1. Create ValidationResult.ts

Define a simple result containing pass/fail status, command, exit code, and useful error output.

2. Create TestRunner.ts

Run the project test command and capture output. Keep command execution controlled and explicit.

3. Create LintRunner.ts

Run the project lint command and capture output.

4. Run TypeScript checking

Run the blueprint command `npm run check-types` after changes.

5. Run lint

Run `npm run lint`. Store the result in the validation result structure.

6. Run relevant tests

Run `npm test` or the appropriate project test command after applying changes.

7. Return failures to Gemini

If validation fails, build a correction context containing the proposed code plus the concrete validation errors.

8. Cap automatic correction

Allow only 2-3 automatic attempts. After the cap, stop and ask the user instead of looping indefinitely.

9. Record the result in memory

Append an event containing the prompt, change summary, status, and validation result.

MVP CHECKPOINT

At this point you have the full core MVP: persistent file memory + AI proposal + real diff + approval + validation + limited correction loop + secure key handling.

Blueprint basis: Phase 6, the 13-step workflow, and the explicit 2-3 attempt guardrail.

13. Phase 7 - Related-File Context

Do this only after Phases 1-6 are stable. The blueprint deliberately avoids whole-repository context in v1.

1. Create RelatedFileResolver.ts

Start with direct imports from the active file.

2. Add type/interface discovery

Include definitions the active file directly uses when practical.

3. Optionally consider recently changed Git files

Use simple-git only if it does not destabilize the MVP.

4. Create ContextFilter.ts

Score/select only genuinely useful neighboring files.

5. Create SecretFilter.ts

Exclude .env, *.pem, *.key, id_rsa, credentials.json, and other configured secret-looking files before model calls.

6. Test token pressure

Confirm that adding related files does not accidentally send the whole repository.

PHASE 7 CHECKPOINT

The active file gets only narrow, relevant neighboring context. Secret-looking files never enter the AI context.

14. Phase 8 - Context Packets and Agent Communication

This phase makes the Qwen/Gemini workflow structured and token-efficient.

1. Create ContextPacket.ts

Represent the request with typed fields rather than a long chat transcript.

```
interface ContextPacket {
  requestId: string;
  workspaceId: string;
  fileId: string;
  filePath: string;
  language: string;
  userPrompt: string;
  currentFile: { content: string; hash: string };
  memory: {
    summary: string;
    constraints: string[];
    decisions: string[];
  };
}
```

```

    knownFlaws: string[];
    recentEvents: MemoryEvent[];
  };
  relatedFiles: RelatedFile[];
  availableTools: string[];
  tokenBudget: number;
}

```

2. Create ArtifactStore.ts

Store reusable artifacts by ID so later agents can refer to an artifact instead of copying its content again.

3. Create AgentMessageBus.ts

Pass structured messages between the orchestrator and providers.

4. Create ProviderRouter.ts

Route cheap context work to Qwen and hard reasoning/code generation to Gemini.

5. Use hot vs cold memory

Send current constraints, unresolved flaws, active decisions, and a few relevant recent events by default. Keep older superseded attempts local.

PHASE 8 CHECKPOINT

Qwen and Gemini communicate through compact structured data, not repeated full conversations. Artifact IDs are used when possible.

15. Phase 9 - Sidebar/Webview UI

Only polish the interface after the underlying workflow is stable.

1. Create the sidebar entry

Register a VS Code webview/sidebar view for the agent.

2. Show current file

Display file name/path and language so the user knows exactly what is in scope.

3. Show memory

Add summary, decisions, constraints, and known flaws to a memory panel.

4. Add chat input

Let the user send a plain-language request without opening the Command Palette every time.

5. Add loading/error states

Show when Qwen/Gemini is working and display actionable errors when a provider fails.

6. Add diff/approval controls

Keep the safety flow obvious: Review Diff -> Approve or Reject.

PHASE 9 CHECKPOINT

The extension feels integrated into VS Code, but the core safety behavior is unchanged.

16. Phase 10 - Packaging, Hardening and Submission

1. Create settings carefully

Expose only safe configuration. Keep API keys in SecretStorage.

2. Verify Workspace Trust

Do not run AI actions in an untrusted workspace.

3. Verify secret filtering

Check that secret-looking files are excluded before any provider call.

4. Check scope enforcement

Attempt a request that mentions another file and verify the extension does not silently modify it.

5. Check logging

Make sure full prompts, API keys, and secrets are not written to logs.

6. Run final quality commands

Run type-check, lint, and tests from a clean terminal.

Type / create exactly this:

```
npm run check-types
npm run lint
npm test
```

7. Package the extension

Create the installable VSIX.

Type / create exactly this:

```
npm run package
```

Expected result: A .vsix file is generated, such as context-aware-extension-0.0.1.vsix.

8. Install from VSIX

In VS Code: Extensions -> ... -> Install from VSIX -> select the generated .vsix.

9. Test the installed extension

Run the end-to-end test in a normal VS Code window, not only the development host.

SUBMISSION CHECKPOINT

You have an installable extension, a README, a reproducible demo path, and the MVP safety rules are still working after packaging.

17. Full End-to-End Test Script

Run this exact scenario before submitting. It proves the architecture instead of merely proving that individual functions run.

Step	Action	What must happen
1	Open a TypeScript file	Extension identifies active file/path/language.
2	Run Explain Current File	A summary is produced and memory is loaded/created.
3	Run Show Memory	The active file has its own memory record.
4	Ask: "Add input validation"	Qwen filters/compresses context; Gemini proposes a change.
5	Inspect AI response	Zod accepts the structure; invalid responses are rejected.
6	Open review diff	Original and proposed versions are visible.
7	Click Reject	The file remains unchanged.
8	Request the same/similar change again	A new proposal is generated.
9	Click Approve	WorkspaceEdit applies the proposed change.
10	Run validation	Type-check/lint/tests execute and results are captured.
11	Force a failure for testing	The failure goes back to Gemini for a correction pass.
12	Observe the correction cap	The extension stops after at most 2-3 attempts.
13	Show memory again	The completed event and validation result are recorded.
14	Restart VS Code	The file memory persists.
15	Package as VSIX	The installable package is produced.
16	Install and repeat one small request	Installed build behaves the same as the development build.

18. Troubleshooting Checklist

Problem	Do this
`yo code` is not recognized	Re-open the terminal after installing yo/generator-code, or verify the

	global npm bin path is available in PATH.
The Extension Development Host does not open	Run the TypeScript type check. Fix the first compile error before changing architecture.
A provider call fails	Test the provider independently first. Confirm the local Qwen service is running or the Gemini credential is available in SecretStorage.
Gemini returns text instead of the expected JSON	Do not write the response. Send it through the Zod schema and reject it when the shape is invalid.
The extension changes the wrong file	Stop. Scope enforcement is broken. Verify fileId/filePath is carried from the active-file reader through the orchestrator, validator, diff, and WorkspaceEdit.
Memory becomes huge	Do not store raw transcripts. Keep recent relevant events and have Qwen compress older memory.
AI receives too much context	Check RelatedFileResolver, ContextFilter, SecretFilter, and token budget. Never default to the entire repo.
Validation loops forever	Hard-cap correction attempts at 2-3 and stop with a user-facing failure state.
VSIX packaging fails	Run npm run compile first. Fix type/bundle errors before rerunning vsce package.
You are running out of time	Freeze new features. Complete Phases 1-6, package the MVP, test it, and submit. The blueprint explicitly defines Phases 1-6 as the MVP.

19. Final Submission Checklist

DO NOT SUBMIT THE FIRST BUILD THAT RUNS

Do one clean install/test of the packaged VSIX. Your demo should show the safe behavior, not just a screenshot of code.

- [] Project opens in VS Code with no TypeScript compile errors.
- [] A user can open a file and get a file summary.
- [] Each file has independent persistent memory.
- [] File memory survives a VS Code restart.
- [] The local Qwen provider can summarize/compress context.
- [] Gemini can produce a structured proposed change.
- [] Zod validates every AI response before a write.
- [] The diff is shown before the file changes.
- [] Reject leaves the file unchanged.
- [] Approve applies the change through WorkspaceEdit.
- [] Type-check/lint/tests run after a change.
- [] Failed validation can trigger no more than 2-3 automatic correction attempts.
- [] Secret-looking files are filtered from model context.
- [] API keys are stored in VS Code SecretStorage, not source control.
- [] Workspace Trust is respected.
- [] The project does not silently modify unrelated files.
- [] README explains setup, architecture, usage, security, and known limitations.
- [] npm run package creates a .vsix file.
- [] The .vsix installs successfully from Extensions -> Install from VSIX.
- [] Final demo has one successful request and one rejected request.

20. 90-Second Demo Script for Submission

When you record your demo or show the project to evaluators, do not explain every folder. Demonstrate the core behavior in order.

```
0:00-0:10  Open VS Code and a small TypeScript file.
0:10-0:20  Run Explain Current File and show the summary.
0:20-0:30  Show the file-specific memory.
```

0:30-0:45 Ask: "Add input validation to this function."
 0:45-0:55 Show the generated diff.
 0:55-1:00 Reject once and show that nothing changed.
 1:00-1:10 Ask again and approve.
 1:10-1:20 Show type-check/lint/test result.
 1:20-1:30 Show updated memory event and mention the 2-3 correction cap.

WHAT TO SAY

"The important part is that the model never edits the file directly. It proposes a structured change, the extension validates it, I review the diff, and only after approval does WorkspaceEdit change the file."

Appendix A - IBM Bob Implementation Handoff (No Manual Coding)

Use the blueprint file names as the implementation map. For each item below, tell IBM Bob to implement the files and preserve the architecture/safety rules. You only run the verification commands.

- A1. Phase 1: implement extension activation, command registration, active-file reading, and a basic output panel. Verify with F5 and the Phase 1 smoke test.
- A2. Phase 2: implement .agent_memory.json, stable file identity, content hashing, memory read/write, and event updates. Verify with two separate files and a VS Code restart.
- A3. Phase 3: implement QwenProvider through Ollama for summarizing, compressing and filtering. Verify that Qwen returns a summary for the active file.
- A4. Phase 4: implement GeminiProvider, secure API-key storage, structured JSON output, Zod validation, retries and errors. Verify that a proposed change is returned.
- A5. Phase 5: implement change validation, diff preview, explicit approval and WorkspaceEdit. Verify both Approve and Reject paths.
- A6. Phase 6: implement type-check/lint/test execution, error capture, and a correction loop capped at 2-3 attempts. Verify with a deliberately failing change.
- A7. Phase 7: add direct-import related-file context and secret filtering. Verify secret-looking files never enter model context.
- A8. Phase 8: add ContextPacket, ArtifactStore, AgentMessageBus and ProviderRouter so Qwen/Gemini exchange compact structured data.
- A9. Phase 9: add the sidebar/webview UI, memory viewer, current-file indicator and diff/approval controls.
- A10. Phase 10: add settings polish, privacy notes, optional telemetry opt-in, package the extension and install the VSIX locally.

Manual coding is explicitly not required for this team workflow. The team is responsible for directing Bob, running commands, testing behavior, reviewing diffs, documenting results, and packaging the finished extension.

Appendix B - Source and Scope

This playbook was prepared from the supplied document "Context-Aware Coding Agent - Project Blueprint", Version 1.0, dated September 26, 2026. Terminology, phase structure, safety model, workflow, dependency list, memory model, provider split, and MVP definition follow that source. Where the source marks a feature as optional or later, this guide keeps it optional/later.

Primary source sections used: 1-3 (concept/architecture/stack), 4 (folder structure), 5 (memory), 6 (AI provider strategy), 7 (agent communication), 8 (13-step workflow), 9 (security), 10 (roadmap + MVP definition), 11 (team plan), 12 (architecture map), 13 (commands/dependencies/risks).

Appendix C - Team Evidence Checklist

- [] YOU: terminal setup + dependency install screenshot
- [] YOU: provider/orchestrator integration result
- [] YOU: final npm run package result
- [] SHUBHAM: memory test with two files + restart proof
- [] SHUBHAM: validation/correction test result
- [] SACHIN: at least one genuine QA test result (connectivity, diff approval, UI or regression)
- [] RAM: README update + final checklist + VSIX installation proof + demo notes

[] Team: one successful change request + one rejected change request

[] Team: no API key in source/logs; secret-looking files filtered

FINAL RULE: WORKING MVP > PRETTY UI > EXTRA FEATURES

Get the safe edit loop working first. Then polish. Do not let the last two hours become a rewrite.